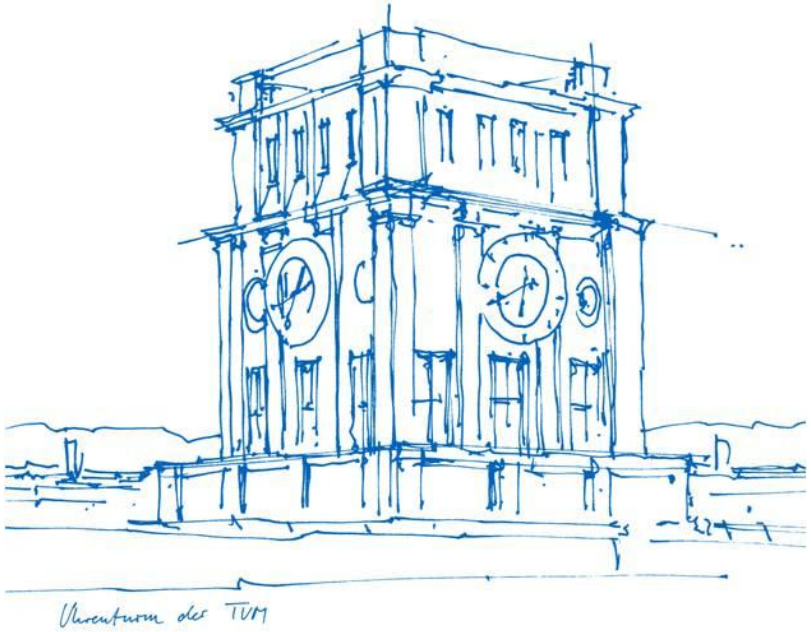
Predicting Patient Length of Stay Using Hospital Admission Data

Presenter: Shuyuan Wang
Beibei Le
Ping-Jui Chen (Rick)
Luoze Guo
Chia-Chen Li



Agenda

Part One:

Exploratory Data Analysis

- Data Cleaning
- Feature Engineering
- Defining Features And Targets
- Data Splitting
- Visualization

Part Two:

Machine Learning

- Linear Regression
- Random Forest
- XGBoost

Introduction

- What is the problem?
- What does it matter?
- What is our goal?
- What is the data?
- What is the target?



Introduction

Problem Statement:

Hospitals need to allocate beds, staff, and resources efficiently, but the length of a patient's hospital stay can vary widely due to multiple medical and personal factors. This **variability** makes it challenging to plan ahead.

Goal:

Predict **Length of Stay(LOS)** using check-in data
Helps hospitals **optimize beds, staff, and outcomes**

Feature Overview

Dataset:

455,000+ patient records

18 features: hospital, patient, admission info

Target: Stay

	🏥 Hospital-related	👤 Patient-related	📝 Admission-related
0	Hospital_code	case_id	Type of Admission
1	Hospital_type_code	patientid	Severity of Illness
2	Hospital_region_code	City_Code_Patient	Bed Grade
3	City_Code_Hospital	Age	Admission_Deposit
4	Available Extra Rooms in Hospital	Visitors with Patient	Stay

Data Cleaning

- Handling Missing Value
- Proxy MCAR Test
- Removing Irrelevant Columns



Handling Missing Values

```
case_id                                0
Hospital_code                          0
Hospital_type_code                     0
City_Code_Hospital                     0
Hospital_region_code                   0
Available Extra Rooms in Hospital      0
Department                             0
Ward_Type                              0
Ward_Facility_Code                     0
Bed Grade                            148
patientid                              0
City_Code_Patient                   6689
Type of Admission                      0
Severity of Illness                    0
Visitors with Patient                  0
Age                                    0
Admission_Deposit                      0
Stay                              137057
dtype: int64
```

1) Found **missing values** in:
- Bed Grade
- City_Code_Patient
- Stay

2) Ran **proxy MCAR test** to assess randomness

Stay:

prediction **target**, any row missing is **useless for training, dropped**

```
df.columns = df.columns.str.strip()
df = df.dropna(subset=['Stay'])
```

Bed Grade & City_Code_Patient:

MCAR test showed **very minor variation, mode imputation**

```
df['Bed Grade'] = df['Bed Grade'].fillna(df['Bed Grade'].mode()[0])
df['City_Code_Patient'] = df['City_Code_Patient'].fillna
 (df['City_Code_Patient'].mode()[0])
```

Dropping Irrelevant Columns

Removed columns with no predictive value:

- serve as unique identifiers – not useful for prediction

```
df = df.drop(columns=['case_id','patientid','Hospital_code',
 'Hospital_region_code','City_Code_Hospital',
 'City_Code_Patient_missing','City_Code_Patient'], errors='ignore')
```

Feature Engineering

- Identifying categorical or non-numeric columns
- Two Encoding Strategies:
- LabelEncoder & OneHotEncoder
- Creating New Columns
- Normalizing Numeric Features



Identifying Categorical or Non-numeric Columns

Two Encoding Strategies:

- **Label Encoding** (ordinal, where order matters):
  `Type of Admission`, `Severity of Illness`, `Age`, `Stay`

- **One-Hot Encoding** (nominal):
  `Hospital_type_code`, `Department`, `Ward_Type`, `Ward_Facility_Code`

Creating New Columns

Stay_Category:

- grouped 11 original stay labels into just **three categories:**

  Short (0–20)

  Medium (21–50)

  Long (more than 50).

- Simplifies model & improves class balance

- Encoded original **`Stay`** from 0 to 10 for flexibility

Age:

- original age = ranges “31-40”
- mapped to **numeric codes** from 0 to 9
- preserved **natural order**, made it **usable** for modeling

Label Encoding

- Helps model recognize **progressive** relationships

Type of Admission:

→ ‘**Urgent’** to 0, ‘**Trauma’** to 1, and **‘Emergency’** to 2

Severity of Illness:

→ **‘Minor’** to 0, **‘Moderate’** to 1, and **‘Extreme’** to 2

Scaling Numeric Features

- Original values ranged from **1,800 to over 11,000**
- Large values can **skew model learning** if unscaled
- Scale it to **[0, 1]**
- Preserved distribution shape while **balancing feature influence**

Admission_Deposit
4911.0
5954.0
4745.0
7272.0
5558.0

Before

Admission_Deposit
0.337858
0.451129
0.319831
0.594266
0.408123

After

Data Preparation for Modeling

- Defining Features and Target Variables
- Splitting Train/Test Sets



Feature and Target Definition

Defining Features (X) and Target (y)

X (Features): All input variables (features) used by the model to make predictions

use the ‘drop’ function to remove the ‘Stay’ and 'Stay_Category' column in order to prevent data leakage:

```
->X = df_one_hot.drop(['Stay', 'Stay_Category'], axis=1)
```

y (Target): The column to be predicted by the model (include: short,medium,long stay)

```
->y = df_one_hot['Stay_Category']
```

Train-Test Split

Function: train_test_split() from sklearn.model_selection

Purpose: Randomly divide the data into training and testing sets

70% of data for training, **30%** for testing

Set random_state=42 to ensure reproducibility

->X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

Visualization



Bar Plot - Stay Category Distribution

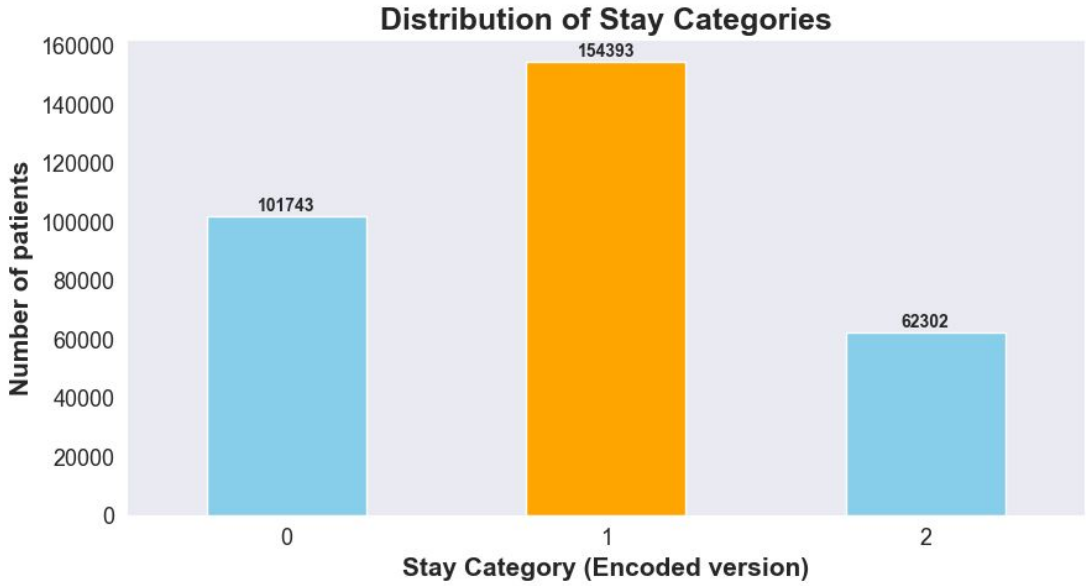
Bar chart shows patient counts per stay category (0: Short, 1: Medium, 2: Long)

Helps check if dataset is imbalanced

Observation:

Medium stay (1) is the majority class

->Dataset is slightly imbalanced, however ,this result consistent with real-world scenario



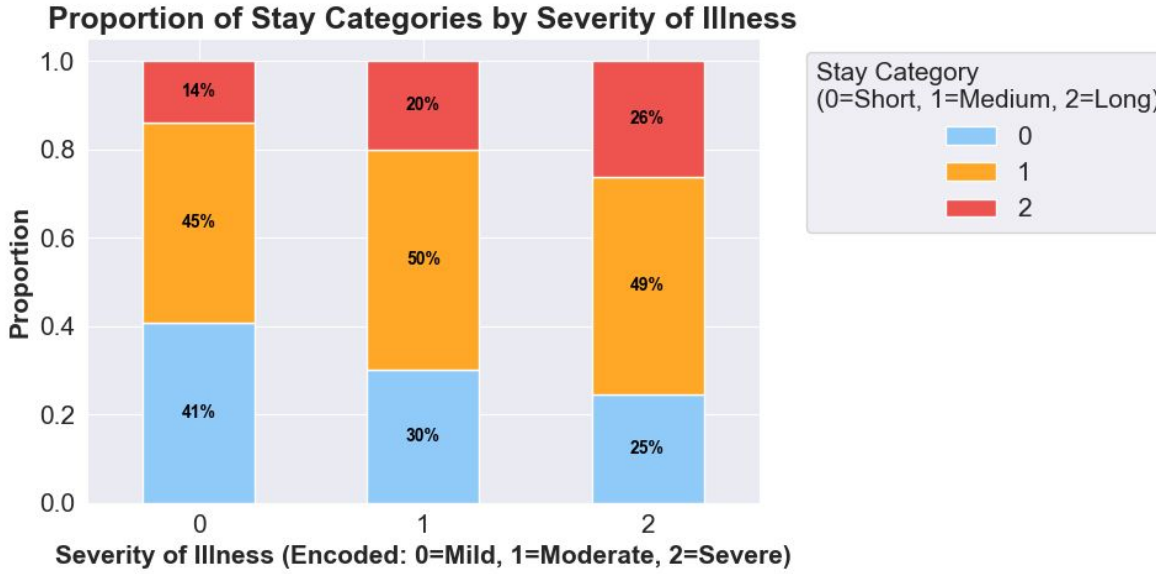
Stacked Bar Plot - Stay Category vs Severity

Encoded:

stay categories:
0=Short, 1=Medium, 2= Long
Severity of illness:
0 = Mild, 1 = Moderate, 2 = Severe

Patterns:

- Long stay proportion ⬆️ with severity ⬆️
- Short stay proportion ⬇️ with severity ⬆️
- Medium stays dominate overall



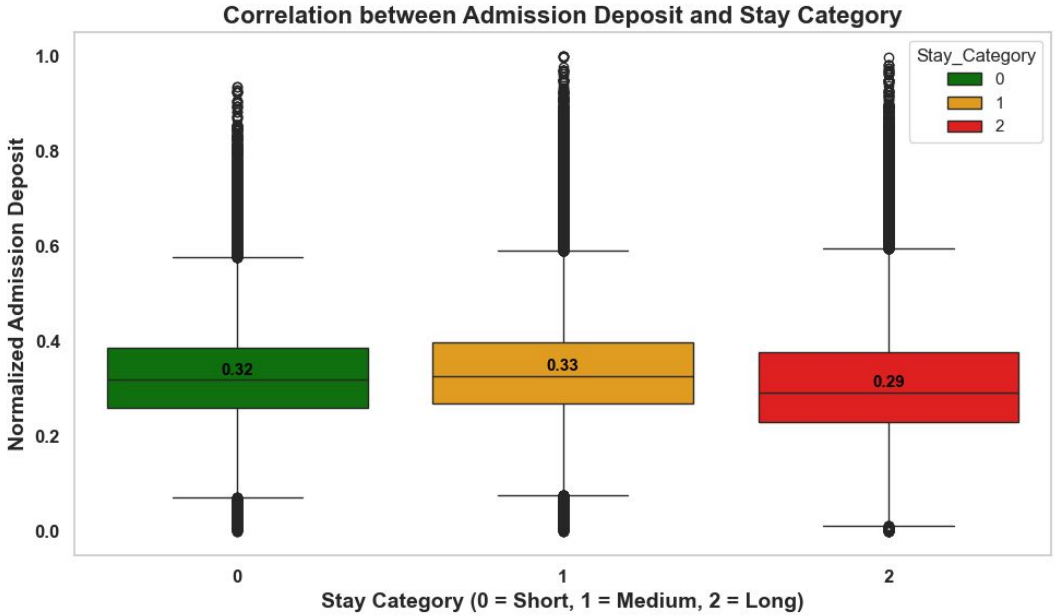
Box Plot - Admission Deposit vs Stay Category

Boxplot shows normalized deposit across stay types

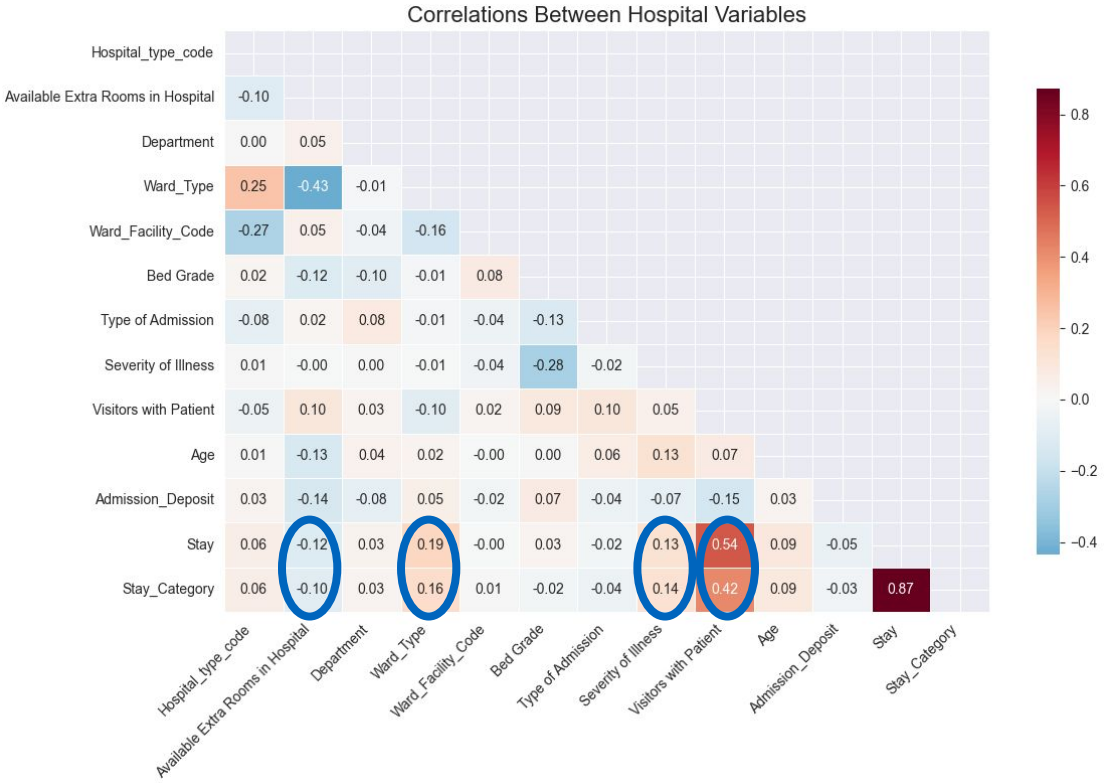
Observation:

- Medians around 0.3 for all
- **Low variance** within each group
- Outliers exist but pattern is not strong

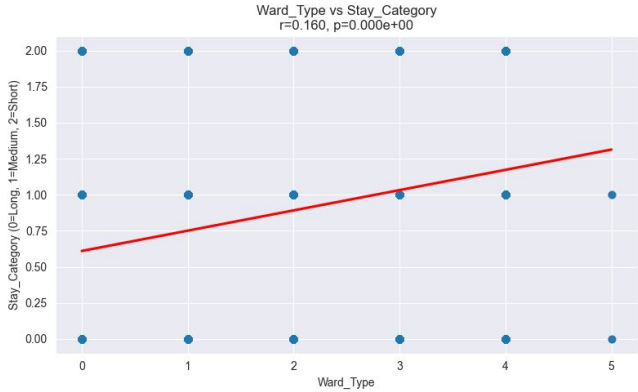
->We suspect admission deposit may not be a strong predictor



Visualization: Correlation HeatMap



- The Correlation HeatMap entails certain linear correlation from specific features and our target



Machine Learning

- Linear Regression
- Random Forest
- XGBoost



Introduction to ML Models

Linear Models

Based on the correlation heatmap to explore the linear correlation

- *Fast check for any linear signal*
- *Coefficients reveal feature influence*

→ Linear Regression

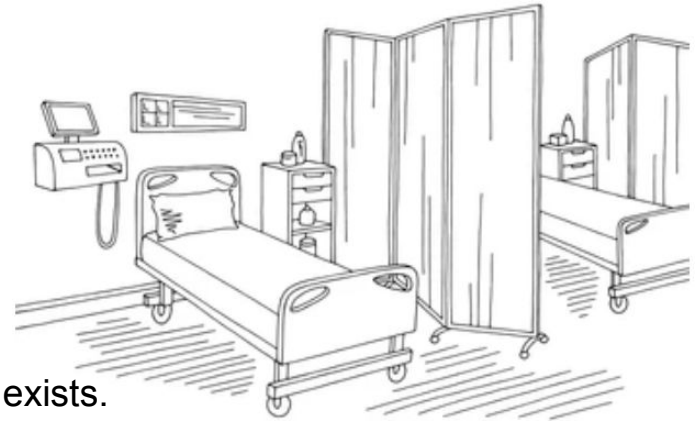
Non-Linear Models

There may be factors among features that affect the result

- *Better suited to discrete classes*
- *Robust to mixed feature types*

→ Random Forest, XGBoost

Linear Regression Analysis



Why begin with Ordinary Least Squares (OLS)?

- **Fast sanity-check** – quick to fit, shows if any linear signal exists.
- **Transparent** – coefficients & p-values tell us which factors lengthen or shorten stay.
- **Baseline benchmark** – gives R^2 & RMSE targets for Random Forest / XGBoost to beat.

source of picture1

Data Preparation & Feature Selection

- Removed rows with missing Stay (target).
- Encoded categorical features: Label Encoding (ordinal) and One-Hot Encoding (nominal).
- Retained variables with |corr(Stay)| > 0.10:
  - Available Extra Rooms in Hospital,
  - Severity of Illness,
  - Visitors with Patient,
  - Ward_Type_Q, Ward_Type_R, Ward_Type_S, Ward_Type_T, Ward_Type_U.

Fitting Key Results and Interpretation

- R-squared: 0.233
- Significant predictors ($p < 0.05$):
  - Ward_Type_S ($\beta \approx 0.468$)
  - Ward_Type_T ($\beta \approx 0.399$)
  - Ward_Type_R ($\beta \approx 0.334$)
  - Visitors with Patient ($\beta \approx 0.176$)
  - Ward_Type_Q ($\beta \approx 0.156$)
  - Severity of Illness ($\beta \approx 0.123$)
  - Available Extra Rooms ($\beta \approx -0.038$)

- Fitted OLS model explains 23% of variance.
- More visitors and higher illness severity are linked to longer stays.
- Certain ward types correlate with extended stays.
- More available rooms slightly reduce stay length, possibly due to capacity effects.

Test Key Results and Interpretation

- Test R^2 = 0.228
  - 22.8 % variance is captured.
- Test RMSE = 0.62
  - Misclassified by one category level after estimation.

- Train R^2 0.23, Test R^2 0.23 → consistent,
- but low → motivates stronger models.

Limitations

- Low R^2 indicates many influencing factors remain unmodelled.
  - Linear model cannot capture nonlinear relationships.

Next steps

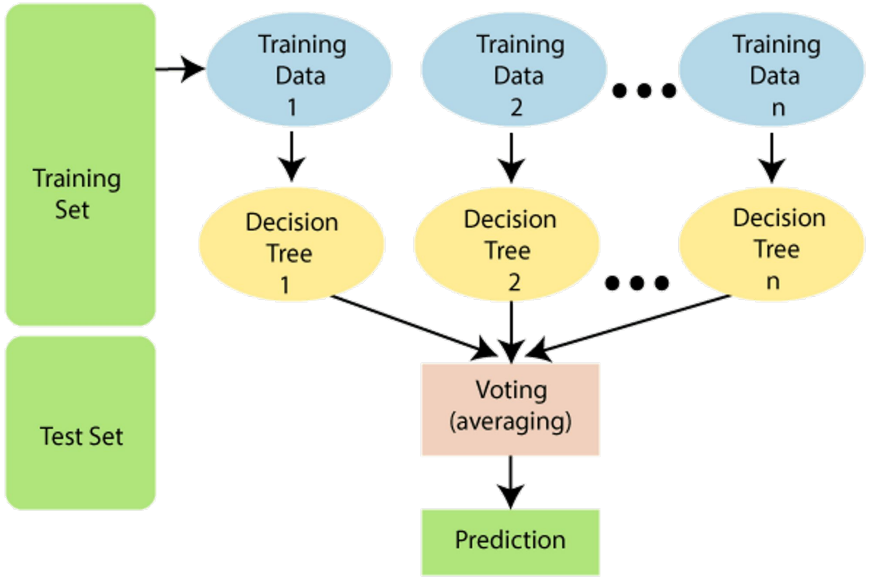
- Trying to explain uncaptured data (~ 77%) , apply non-Linear model.

Choose Model: Random Forest

- Ensemble Learning: Train multiple decision trees on different bootstrapped samples.

Key Advantages

1. Captures non-linear relationships and complex interactions.
2. Robust to outliers and noise.
3. Provides feature importance, helping identify the most influential variables.



Source: https://www.almabetter.com/bytes/tutorials/data-science/random-forest

Training Result: Using Optuna Optimization

Optuna is a **hyperparameter optimization framework** for machine learning. It uses **efficient search algorithms** to find the best model parameters automatically.

Accuracy: ~61%

- only recalls 40% of class 0
- class 1, 2 have more precision and recall

```
classifier = RandomForestClassifier(
    n_estimators=980,
    max_depth=17,
    min_samples_split=10,
    min_samples_leaf=13,
    max_features='sqrt',
    criterion='entropy',
    bootstrap=False,
    random_state=0
)
```

```
Accuracy: 0.6085667629694762
              precision    recall  f1-score   support

           0       0.58      0.40      0.47     25436
           1       0.60      0.71      0.65     38598
           2       0.67      0.69      0.68     15576

    accuracy                           0.61     79610
   macro avg       0.61      0.60      0.60     79610
weighted avg       0.61      0.61      0.60     79610
```

Choose Model: XGBoost

- Can handle non-linear relationships and categorical data
- Gradient Boosted
- How it works:
  - Initialization
  - Compute Residuals
  - Fit a New Tree
  - Update Predictions
  - Repeat the Process



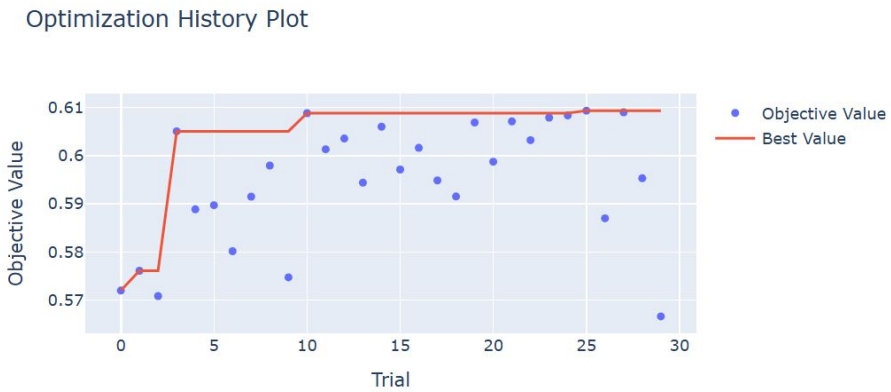
Source: https://xgboost.readthedocs.io/en/stable/

Optimization: Optuna

Features of XGBoost

- Uses feature subsampling
- Supports regularization to prevent overfitting

```
'n_estimators': trial.suggest_int('n_estimators', 100, 600),
'max_depth': trial.suggest_int('max_depth', 3, 50),
'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
'subsample': trial.suggest_float('subsample', 0.5, 1.0),
'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
'gamma': trial.suggest_float('gamma', 0, 0.5),
'reg_alpha': trial.suggest_float('reg_alpha', 0.0, 1.0),
'reg_lambda': trial.suggest_float('reg_lambda', 0.0, 2.0),
```



Result

Accuracy: **61%**

Class 1, 2 (medium, long term stay) predicted best

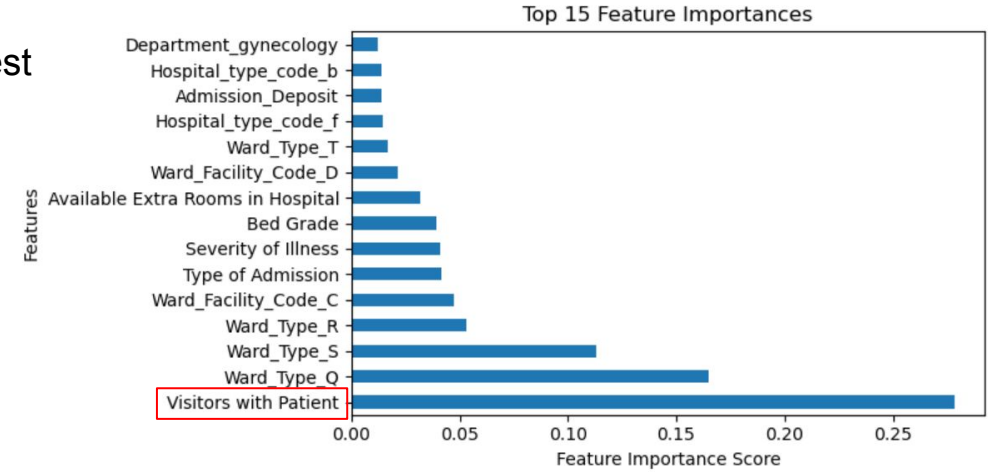
Still room to improve on **Class 0** (short term)

```
Accuracy:  60.925763095088556

Classification Report:
              precision    recall  f1-score   support

           0       0.57      0.43      0.49     25436
           1       0.60      0.70      0.65     38598
           2       0.68      0.67      0.67     15576

    accuracy                           0.61     79610
   macro avg       0.62      0.60      0.60     79610
weighted avg       0.61      0.61      0.60     79610
```



Most important feature: Visitors with Patient

Conclusion

- Random Forest and XGBoost significantly outperform Linear Regression
- Especially effective in predicting medium and long stays
- Features like "Visitors with Patient" are strong predictors of stay duration

Thank you for listening.

Q&A session

